Delete code with evidence: dead symbols, duplicate logic and external entry points

Researched 2026-10-20.

The useful workflow

The reel recommends finding dead code, duplicate logic and abandoned routes before deleting in small batches. Its most useful constraint is to protect externally invoked entry points. A webhook, scheduled job, plugin export or auth callback can be active without a call site in the repository.

A correction to the dossier

The Muse dossier says the reel's "four agents" claim is wrong and cites older descriptions of three reviewers. **Current official documentation now describes four parallel cleanup reviewers for /simplify.** It distinguishes cleanup from /code-review's correctness checks. Use the installed command menu and current docs instead of carrying the dossier's correction forward. Claude Code command reference (https://code.claude.com/docs/en/commands).

```
/simplify
/code-review
```

These start future model work; they were not run against a user's repository during this research. GitClear's duplication figures are observational research, not proof that every generated line should be removed. The original report (https://gitclear-public.s3.us-west-2.amazonaws.com/GitClear-AI-Copilot-Code-Quality-2025.pdf) is the place to inspect dataset definitions and projections.

Static checks before model suggestions

In an existing JavaScript or TypeScript project, Knip documents Node.js 20.19+ (or Bun), setup through its configuration initializer and a report command. Review configuration changes before keeping them. Knip setup (https://knip.dev/overview/getting-started).

```
npm init @knip/config
npm run knip
```

For Python, use a disposable virtual environment or the project's own environment. Vulture's high-confidence mode narrows the report; it does not prove runtime non-use. Vulture documentation (https://github.com/jendrikseipp/vulture).

```
python -m pip install vulture
vulture . --min-confidence 100
```

A static report is a candidate list. Dynamic imports, framework conventions, reflection, command discovery and deployment hooks need additional evidence.

Five practical prompts

1. Inventory candidates with symbol references, re-exports and framework entry points; do not delete.
2. Compare repeated logic by behavior and callers before proposing one shared implementation.
3. Inventory routes, their authentication and both internal and external callers.
4. Review the current diff for unnecessary complexity without changing public behavior.
5. Protect the explicit external-entry-point list until a human confirms retirement.

For each proposed deletion, require an evidence row: file or symbol, why it looks unused, external-entry-point check, verification command, and rollback plan. "No text search matches" alone is insufficient.

Reusable prompt

```
Audit this repository for removable code without editing anything.
Inventory dynamic entry points, scripts, scheduled work, webhooks, CLI commands,
plugin discovery and callbacks before flagging dead code.
For every candidate, cite references checked, expected behavior and a test.
Propose one small batch with a reversible plan. Do not stage all files,
delete external entry points or publish changes as part of the audit.
```

Project: deletion evidence ledger

Build a local report that joins Knip or Vulture candidates with an allowlist of entry points and a reviewer decision. Start with one language. Export a Markdown table and JSON decisions; never wire deletion into the first version. Add a fixture containing an externally called handler that has no internal references and verify it remains protected.

Operating sequence

Inspect the working tree and preserve an intentional checkpoint. Run existing checks for a baseline, review one candidate batch, make the approved change, then rerun the behavior checks that cover it. Review the actual diff before any commit or deployment. Lower line count is not the acceptance criterion; preserved behavior with less maintenance is.

Original sources

- https://www.instagram.com/reel/Deb0R3wt9el/